Programming Project Report

Warren Roberts
100450116

Problem Statement:

The goal of this project was to learn how to use OpenGL for creating texture-mapped objects that are animated in some manner. We were focused on reading JPEG images and to do so I used a lot of source code for John Gauch's source folder. Normal inputs to the program involve images that are read, and mouse input to interact with the system. Object velocity and angles were used to add motion and animation that the user can view. The program outputs a window with a basketball court, a brick cube, and a steel sphere. Then when the user clicks and drags a new output appears; first the line indicating the trajectory, then the basketballs when the mouse is released. Lastly error handling was done primarily with print statements that output the stats related to the event taking place. In addition, safeguards were added to prevent dividing by zero and going out of bounds. A config file was used so that modifying these bounds is easy.

Design:

The design was based on strict OOP structure to separate responsibilities across different data structures. I have an app directory hosting the main loop and calls to each OpenGL method required. I used unique pointers to apply polymorphism from my base class Object3D to its subclasses like Sphere, Cube, and BasketBall. A vector of these unique pointers was used to store all objects that are displayed in the scene. The main algorithms used for textures was taken from source code and implemented in my structure. Then update functions for BasketBall and other objects were implemented to use delta time to enforce physics and create a clean animation. Cons include at times the project felt very abstract, and at times the meaning was lost, so I commented enough so I could understand everything at a quick glance. Pros include a great result that matches the project description and requirements. I enjoy being able to hot swap textures by just inserting images into a directory and changing the name of its input.

Implementation:

I began by copying my project 3 and adding /libim and /jpeg to my /core directory. This gave me a great starting point. After this I got more source code for cubes and spheres from the professor. With all of this I was able to get models loading extremely fast. The next issue was finding a scene to make. I started with cannonballs and the idea was to launch them from a cannon that could be controller. That did not happen. Instead, I implemented basketball to decrease scope as time was running thin. With the basketball I used what I made for the cannon ball and was able to launch balls from where I drag my mouse. Then I made bounds on the screen and removed WASD movement to keep things simple. I began working on this project on March 26th and

finished on April 3rd, taking me a total of 8 days. I wish I had started earlier to give myself more time to fully implement a movable cannon that fires.

AI Usage Log

Entry 1 — Mouse Dragging for Cannon Launch

Prompt:
"Teach me how to implement mouse dragging in OpenGL… detect distance between click and release…"

Summary of Assistance:

- Learned how to track mouse press, drag, and release using GLUT callbacks
- Stored initial click position and release position
- Computed direction vector and drag distance
- Normalized the direction and scaled it to simulate launch power

How It Was Used:
Implemented a click-and-drag system where the user controls both direction and force of the projectile. This became the core interaction mechanic for launching cannonballs.

Entry 2 — Gravity and Projectile Motion

Prompt:
"How do I implement gravity in updateObject and structure launchProjectile?"

Summary of Assistance:

- Learned how to update position using velocity
- Applied gravity as a constant acceleration on the y-axis
- Structured a `launchProjectile` function to initialize velocity based on drag input
- Added ground collision detection

How It Was Used:
Built a working projectile physics system where cannonballs follow a realistic arc and stop or bounce when hitting the ground.

Entry 3 — Textured Ground and Collision

Prompt:
"How do I make a grass image be the ground and despawn on impact?"

Summary of Assistance:

- Learned how to render a textured quad for the ground
- Understood separation between visual rendering and physics collision
- Implemented collision detection using a defined ground height
- Added optional radius-based collision correction

How It Was Used:
Created a visual ground layer using a grass texture and implemented logic to remove cannonballs when they hit the ground.

Entry 4 — Camera Rotation Controls

Prompt:
"How do I rotate the camera left, right, up, and down with keys?"

Summary of Assistance:

- Introduced yaw (horizontal rotation) and pitch (vertical rotation)
- Converted angles into a direction vector
- Used `gluLookAt` to control camera orientation
- Added clamping to prevent camera flipping

How It Was Used:
Implemented keyboard-controlled camera rotation, allowing the user to explore the scene dynamically.

Entry 5 — Fixed Camera Positioning

Prompt:
"How do I set a good camera position and angle to watch the scene?"

Summary of Assistance:

- Learned how to position camera for optimal viewing (offset, height, distance)
- Targeted a focal point slightly above the ground for better framing

- Used `gluLookAt` to create a cinematic perspective

How It Was Used:
Set a clean, fixed camera view to showcase the cannonball motion and environment effectively.

Entry 6 — Drag Line Visualization

Prompt:
"How do I draw a line while dragging the mouse?"

Summary of Assistance:

- Tracked current mouse position during drag
- Rendered a line between start and current positions
- Converted screen coordinates to world coordinates
- Disabled lighting and depth testing for visibility

How It Was Used:
Added a visual aiming line to show direction and strength of the cannon launch, improving usability and clarity.

Entry 7 — Wall Bounce Physics

Prompt:
"How do I make cannonballs bounce off window borders that change with A/D?"

Summary of Assistance:

- Implemented boundary collision detection
- Reflected velocity on impact with walls
- Added damping factor for realistic energy loss
- Adjusted wall positions dynamically based on camera/view

How It Was Used:
Enabled cannonballs to bounce off screen edges, making the simulation more dynamic and visually interesting.

Entry 8 — On-Screen Text Rendering

Prompt:
"How do I add text to the screen?"

Summary of Assistance:

- Used GLUT bitmap text functions
- Switched to orthographic projection for UI rendering
- Disabled depth testing and lighting for text clarity

How It Was Used:
Displayed instructions and debug information directly on screen.

Entry 9 — Basketball Rotation (Realistic Spin)

Prompt:
"What is a realistic rotation method for the basketball?"

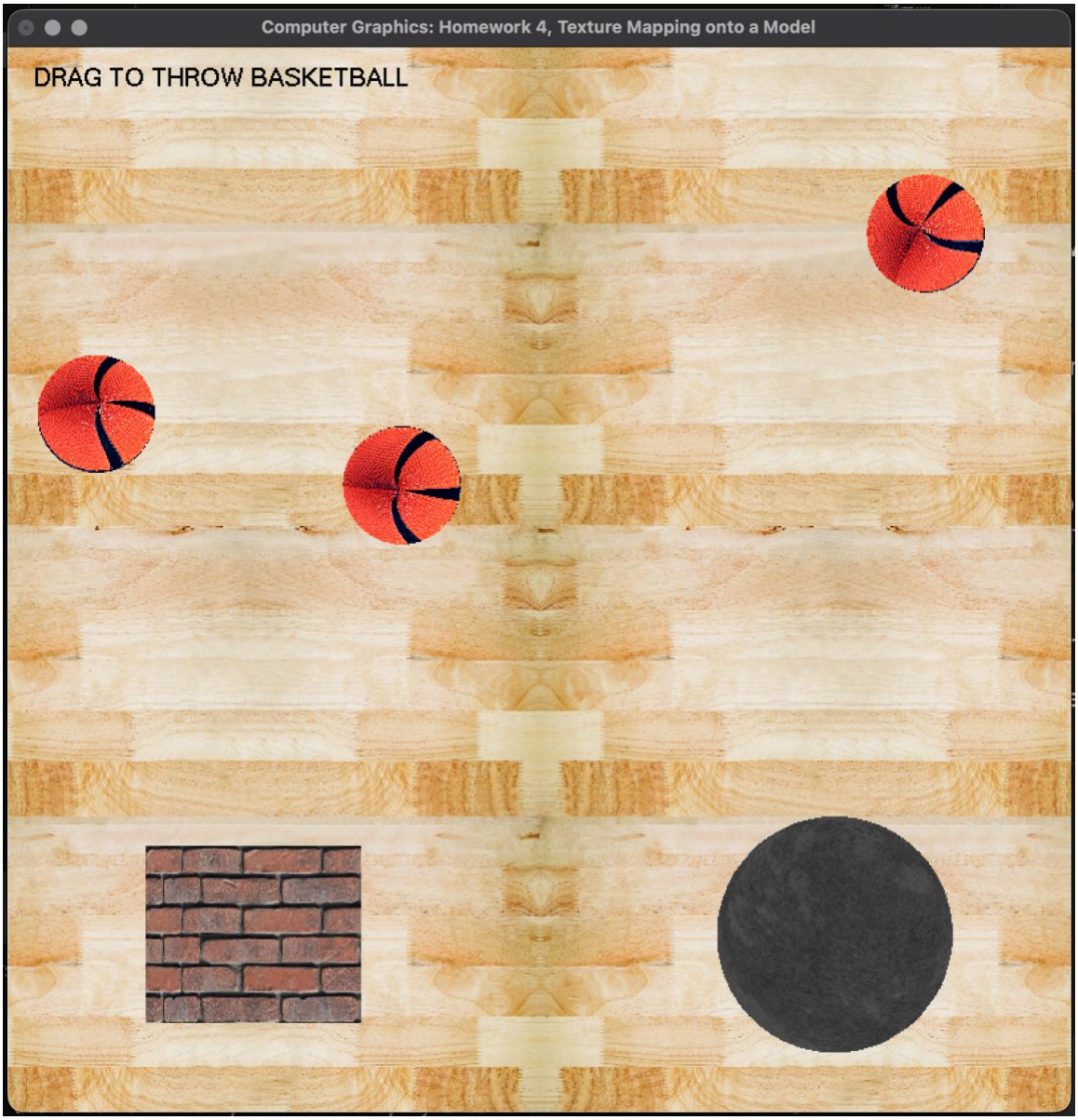
Summary of Assistance:

- Introduced angular velocity separate from linear motion
- Linked spin to launch speed for realism
- Updated rotation over time with optional damping
- Applied rotation using transformation matrices

How It Was Used:
Implemented realistic spinning motion for the basketball, improving visual quality and physical believability.

Testing:

The program was tested incrementally as it was developed to make sure each feature was fully implemented before moving on. Initial tests started with rendering a cube and sphere. After this the texture was loaded and applied to those models. This was done before object types and any real structure was made. This made it easy to ensure I had everything wired together correctly and allowed me to move everything and refactor piece by piece. Normal inputs include JPEG images and dragging the mouse to get a trajectory and launch a ball. Special cases included bounds checking for many components. Specifically the walls, I had trouble lining up where the walls were and when the ball should bounce. Then I realized the ball was fast so I added a reduced velocity upon bouncing. After ensuring everything worked well and with the refactoring I checked the textures loaded on each object. Everything worked as expected in the end, but it took a lot of testing and time to get it that way.

Conclusions:

This project was a success, and I am happy with the outcome. It would be cool to add more interactable objects in the view! If I could do this again, I would have started earlier and been able to implement a movable cannon that when clicking and dragging launches a cannonball. Collisions between objects would be cool too (if I left the model texture as a basketball and had them bounce off each other). The project took from March 26th to April 3rd, so a total of 8 days with some breaks. I spent a lot of time in the end changing the design because initially I started with the idea of launching cannonballs from a cannon but decided on just throwing basketballs from where the mouse is.